

SymPy Bug Report

1 Bug 23: solveset returns 0 for $\operatorname{atan}(x) = \pi$ outside the principal atan range

1.1 Minimal Reproducer

```
from sympy import Eq, S, N, atan, pi, solveset, symbols

x = symbols("x")
eq = Eq(atan(x), pi)
sol = solveset(eq, x, domain=S.Complexes)

print("solution =", sol)
for s in sol:
    print("residual at", s, "=", N(atan(s) - pi, 80))
```

1.2 Observed Behavior

```
solution = {0}
residual at 0 =
-3.1415926535897932384626433832795028841971693993751058209749445923078164062862090
```

SymPy returns the candidate $x = 0$. Substituting that value into the original equation gives

$$\operatorname{atan}(0) - \pi = -\pi,$$

so the returned value does not satisfy the equation.

1.3 Expected Behavior

The correct solution set over $S.Complexes$ is $\emptyset$. The principal value of $\operatorname{atan}(0)$ is 0, not π , and the principal branch of atan does not attain the real value π .

1.4 Mathematical Explanation

The equation being solved is

$$\operatorname{atan}(x) = \pi.$$

Applying $\tan$ to both sides gives the tempting candidate

$$x = \tan(\pi) = 0.$$

That inference is invalid for the principal inverse tangent. The identity $\tan(\operatorname{atan}(x)) = x$ holds where the principal arctangent is defined, but $\operatorname{atan}(\tan(y))$ does not equal y for arbitrary y . Instead, atan returns a principal value with real part in the principal arctangent strip. In particular,

$$\operatorname{atan}(0) = 0,$$

so $x = 0$ satisfies $\operatorname{atan}(x) = 0$, not $\operatorname{atan}(x) = \pi$. The reported answer is therefore an extraneous solution introduced by treating a principal-branch inverse as though it were a globally valid inverse.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant solver code in `sympy/solvers/solveset.py`, specifically `_solveset` and `_invert_complex`. The traced call path starts from the complex-domain call to solve $\operatorname{atan}(x) = \pi$. The equation is normalized to `atan(x) - pi`. This expression is not handled by `_solve_trig`, so `_solveset` reaches the generic inversion call. The inverter removes the additive $-\pi$, obtains `atan(x) = pi`, and dispatches to the generic inverse-function branch in `_invert_complex`.

The key rule in `_invert_complex`, located at `sympy/solvers/solveset.py:550-557`, is:

```
if hasattr(f, 'inverse') and f.inverse() is not None and \
    not isinstance(f, TrigonometricFunction) and \
    not isinstance(f, HyperbolicFunction) and \
    not isinstance(f, exp):
    ...
    return _invert_complex(f.args[0],
                           imageset(Lambda(n, f.inverse()(n)), g_ys), symbol)
```

For $f = \operatorname{atan}(x)$, the inverse method returns $\tan$, with the inverse defined in `sympy/functions/elementary/trigonometric.py:2775`. The diagnosis notes that inverse trigonometric functions are not excluded by this branch. With target set $\{\pi\}$, the branch maps the target through $\tan$, producing $\{\tan(\pi)\} = \{0\}$. The finite candidate is then accepted because the original expression is defined at $x = 0$; no residual equality check rejects it. Thus the solver returns a solution of $\tan(\operatorname{atan}(x)) = \tan(\pi)$, not a solution of the original principal-value equation.

A plausible fix direction supported by the diagnosis is to treat inverse trigonometric functions as partial inverses. The solver could invert them only when the target is known to lie in the appropriate principal range, or it could post-filter finite candidates against the original equation.

1.6 Additional Failing Instances

The same mechanism appears for other real multiples of π outside the principal range of atan . Applying $\tan$ to each target produces 0, but the principal value $\operatorname{atan}(0)$ is 0, not the requested nonzero multiple of π . The independent verification records the following parametrized cases.

Input / Expression	SymPy behavior	Expected behavior
$\text{atan}(x) = \pi$	returns a set containing 0	$\emptyset$
$\text{atan}(x) = -\pi$	returns a set containing 0	$\emptyset$
$\text{atan}(x) = 2\pi$	returns a set containing 0	$\emptyset$
$\text{atan}(x) = -2\pi$	returns a set containing 0	$\emptyset$
$\text{atan}(x) = 3\pi$	returns a set containing 0	$\emptyset$
$\text{atan}(x) = -3\pi$	returns a set containing 0	$\emptyset$

The expected behavior is the same across the family because each target is a real value outside the real principal range of atan, while the returned candidate is only a solution of the tangent equation obtained after an invalid global inversion step.

1.7 Related Issues Assessment

The available deduplication evidence found related background on principal branches of inverse trigonometric functions and the SymPy `solveset` contract, but no public report of this exact reproducer or output. The verdict is therefore a new instance of a known failure mode: the broad issue of respecting principal branches is known, but this exact atan case with the false result $\{0\}$ appears previously unreported and remains individually actionable as a focused issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import Eq, S, atan, pi, solveset, symbols, simplify

x = symbols("x")

CASES = [
    (pi, 0),
    (-pi, 0),
    (2*pi, 0),
    (-2*pi, 0),
    (3*pi, 0),
    (-3*pi, 0),
]

@pytest.mark.parametrize("target,bad", CASES)
def test_solveset_atan_rejects_values_outside_principal_range(target, bad):
    sol = solveset(Eq(atan(x), target), x, domain=S.Complexes)

    assert sol == S.EmptySet
```

```
assert simplify(atan(bad) - target) != 0
```